

PyTorch DataLoaders & Exploratory Data Analysis

Module 3 of 4 — Python and Machine Learning with PyTorch

Sayan CHAKI

Chercheur

LIRIS, Ecole Centrale Lyon, Univ. Lyon 2, INSA

September 9, 2026

Table of Contents

- 1 Dataset and DataLoader
- 2 Building Custom Datasets
- 3 Exploratory Data Analysis
- 4 Practical Session

The problem we are solving today

In Module 2 we trained on a tensor that fit entirely in memory:

```
X = torch.randn(64, 4)
```

Real datasets do not cooperate:

- ImageNet is 150 GB of JPEG files. It will not fit in RAM.
- Samples must be **shuffled** differently at every epoch.
- Images need **decoding and augmentation** on the fly.
- Loading must happen **in parallel** with GPU computation, otherwise the GPU sits idle.

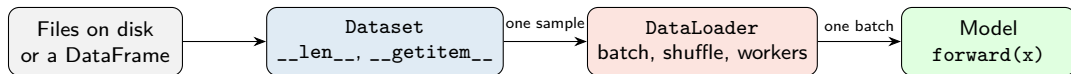
PyTorch's answer

Separate **what a sample is** (Dataset) from **how samples are served** (DataLoader).

Where we are

- 1 Dataset and DataLoader
- 2 Building Custom Datasets
- 3 Exploratory Data Analysis
- 4 Practical Session

The two-object design



- Dataset answers: *how many samples are there, and how do I fetch sample number i ?*
- DataLoader answers: *in what order, in what group size, and with how many worker processes?*

The Dataset contract: exactly two methods

```
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, ...):
        # Store paths, load a CSV, define transforms.
        # Do NOT load the whole dataset into memory here if it is large.
        ...

    def __len__(self):
        """Total number of samples."""
        return self.n

    def __getitem__(self, idx):
        """Return ONE sample: usually a (features, label) tuple."""
        x = ...      # a tensor
        y = ...      # an int or a tensor
        return x, y
```

Recall Module 2

These are the dunder methods `__len__` and `__getitem__`. Nothing new: a `Dataset` is just a Python object that supports `len(ds)` and `ds[i]`.

DataLoader: the important arguments

```
from torch.utils.data import DataLoader

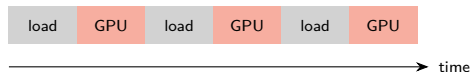
loader = DataLoader(
    dataset,
    batch_size=32,          # samples per batch
    shuffle=True,           # reshuffle every epoch (train only, never for test)
    num_workers=2,         # subprocesses that prefetch data in parallel
    pin_memory=True,       # faster host-to-GPU copy when using CUDA
    drop_last=False,       # drop the final smaller batch if True
)

for batch_x, batch_y in loader:      # the loader is ITERABLE
    print(batch_x.shape, batch_y.shape)
    break                            # torch.Size([32, 1, 28, 28]) ...
```

- `len(loader)` is the number of **batches**, not samples.
- Default collation stacks samples along a new dimension 0.

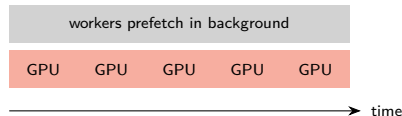
Why performance depends on this

Without workers



The GPU waits while the CPU decodes images.

With `num_workers > 0`



Loading overlaps computation: same work, much less wall time.

In Colab

Use `num_workers=2`. Higher values often crash the hosted runtime.

Where we are

- 1 Dataset and DataLoader
- 2 Building Custom Datasets**
- 3 Exploratory Data Analysis
- 4 Practical Session

Images, the easy way: ImageFolder

If your data is organised by folder, one line is enough.

```
data/  
  train/  
    cat/  img001.jpg  img002.jpg  ...  
    dog/  img101.jpg  img102.jpg  ...
```

```
from torchvision import datasets, transforms  
  
tf = transforms.Compose([  
    transforms.Resize((64, 64)),  
    transforms.ToTensor(),           # PIL -> tensor, scales to [0,1]  
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3),  
)  
  
ds = datasets.ImageFolder(root="data/train", transform=tf)  
print(ds.classes)      # ['cat', 'dog'] -> labels 0 and 1, alphabetical  
print(ds[0][0].shape)   # torch.Size([3, 64, 64])
```

Order of transforms

ToTensor() must come before Normalize(), because Normalize operates on tensors, not on PIL images.

Images, the general way: a custom class

```
import os
from PIL import Image
from torch.utils.data import Dataset

class ImageDataset(Dataset):
    def __init__(self, root, transform=None):
        self.transform = transform
        self.classes = sorted(os.listdir(root))
        self.cls_to_idx = {c: i for i, c in enumerate(self.classes)}
        self.samples = [] # store PATHS, not pixels
        for c in self.classes:
            folder = os.path.join(root, c)
            for f in sorted(os.listdir(folder)):
                self.samples.append((os.path.join(folder, f), self.cls_to_idx[c]))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB") # loaded lazily, one at a time
        if self.transform:
            img = self.transform(img)
        return img, label
```

Tabular data: a CSV Dataset

```
import pandas as pd
import torch
from torch.utils.data import Dataset

class CSVDataset(Dataset):
    def __init__(self, csv_path, feature_cols, target_col, normalize=True):
        df = pd.read_csv(csv_path)
        X = torch.tensor(df[feature_cols].values, dtype=torch.float32)
        y = torch.tensor(df[target_col].values, dtype=torch.long)
        if normalize:
            self.mu, self.sigma = X.mean(0), X.std(0)
            X = (X - self.mu) / (self.sigma + 1e-8)  # epsilon avoids /0
            self.X, self.y = X, y

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

Normalisation statistics

Compute mean and std on the **training split only**, then reuse them for validation and test. Otherwise you leak information.

Splitting and reusing a dataset

```
from torch.utils.data import random_split, DataLoader

n = len(dataset)
n_train = int(0.8 * n)
g = torch.Generator().manual_seed(42)           # reproducible split
train_ds, val_ds = random_split(dataset, [n_train, n - n_train], generator=g)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_ds,   batch_size=64, shuffle=False)
```

Two habits worth keeping

- `shuffle=True` for training, `shuffle=False` for evaluation.
- A larger batch size is fine at evaluation time: no gradients are stored.

Where we are

- 1 Dataset and DataLoader
- 2 Building Custom Datasets
- 3 Exploratory Data Analysis**
- 4 Practical Session

Why EDA before modelling

Never train on data you have not looked at. Five minutes of EDA prevents hours of debugging.

- **Shape:** are the dimensions what you expect? Is the channel dimension first or last?
- **Range:** is the data in $[0, 1]$, in $[0, 255]$, or already standardised?
- **Balance:** how many samples per class? A 95/5 split makes accuracy meaningless.
- **Missing values:** any NaN silently poisons the loss.
- **Scale:** features with very different magnitudes dominate distance-based methods such as KNN, which is Module 4.

Inspecting tensors

```
x, y = next(iter(train_loader))      # grab one batch

print(x.shape)                       # torch.Size([32, 1, 28, 28])
print(x.dtype)                       # torch.float32
print(x.min(), x.max())              # check the value range
print(x.mean(), x.std())

# Per-channel statistics over a batch of images (N, C, H, W)
print(x.mean(dim=[0, 2, 3]))         # one mean per channel
print(x.std(dim=[0, 2, 3]))          # one std per channel

# Class balance
print(torch.bincount(y))             # count of each label

# Sanity checks
print(torch.isnan(x).any())          # should be False
print(torch.isinf(x).any())          # should be False
```


Visualising a batch of images

```
import matplotlib.pyplot as plt
from torchvision.utils import make_grid

images, labels = next(iter(train_loader))
grid = make_grid(images[:16], nrow=4, normalize=True, padding=2)

# make_grid returns (C, H, W); matplotlib wants (H, W, C)
plt.figure(figsize=(5, 5))
plt.imshow(grid.permute(1, 2, 0).cpu().numpy())
plt.axis("off")
plt.title(f"labels: {labels[:16].tolist()}")
plt.show()
```

The permute you will forget

PyTorch uses channels-first (C,H,W); matplotlib and NumPy use channels-last (H,W,C).
Always `.permute(1, 2, 0)` before `imshow`.

EDA for tabular data

```
# Pandas for a quick overview
df.head()
df.describe()           # count, mean, std, min, quartiles, max
df.isna().sum()         # missing values per column
df["species"].value_counts()

# The same statistics in PyTorch, for consistency with the training pipeline
X = torch.tensor(df[feature_cols].values, dtype=torch.float32)
print(X.mean(dim=0))    # column means
print(X.std(dim=0))     # column standard deviations

# Correlation matrix with pure tensor operations
Xc = X - X.mean(dim=0)
cov = (Xc.T @ Xc) / (X.shape[0] - 1)
std = X.std(dim=0)
corr = cov / (std[:, None] * std[None, :])
```

Note the broadcasting

`std[:, None] * std[None, :]` builds the outer product of shape (d, d) from a vector of shape $(d,)$, using the rules from Module 1.

Useful matplotlib plots for EDA

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))

# 1. Histogram of one feature
axes[0].hist(X[:, 0].numpy(), bins=30)
axes[0].set_title("feature 0 distribution")

# 2. Scatter of two features, coloured by class
axes[1].scatter(X[:, 0], X[:, 1], c=y, cmap="viridis", s=18)
axes[1].set_title("feature 0 vs feature 1")

# 3. Class balance
axes[2].bar(range(len(torch.bincount(y))), torch.bincount(y).numpy())
axes[2].set_title("samples per class")

plt.tight_layout(); plt.show()
```

An EDA checklist you can reuse

- 1 `len(dataset)` and `len(loader)`: do the numbers add up?
- 2 Pull one batch and print `shape`, `dtype`, `device`.
- 3 Print `min`, `max`, `mean`, `std`.
- 4 Check for `NaN` and `Inf`.
- 5 Count labels per class.
- 6 Plot a grid of raw samples and read the titles: do image and label agree?
- 7 Plot a histogram per feature, or per channel for images.
- 8 Only then write the training loop.

Where we are

- 1 Dataset and DataLoader
- 2 Building Custom Datasets
- 3 Exploratory Data Analysis
- 4 Practical Session**

Colab Notebook 3: DataLoaders, EDA, KNN & K-Means

Part A — Images

- Download MNIST with `torchvision.datasets`, apply transforms.
- Wrap it in a `DataLoader`, iterate, inspect batch shapes.
- Write a custom `Dataset` class over the same data and confirm identical output.

Part B — CSV

- Load the Iris dataset into `pandas`, save it as a CSV, build a `CSVDataset`.
- Normalise features with training-split statistics only.

Part C — EDA

- Visualise a grid of images with `make_grid`.
- Compute per-channel and per-column statistics with tensor operations.
- Build a correlation matrix without leaving PyTorch.

Key takeaways

- 1 Dataset defines **one sample**; DataLoader defines **how batches are served**.
- 2 A custom Dataset needs only `__len__` and `__getitem__`.
- 3 Store file paths in `__init__` and load lazily in `__getitem__`.
- 4 `shuffle=True` for training only; `num_workers` keeps the GPU busy.
- 5 Images are (N,C,H,W) in PyTorch and (H,W,C) in matplotlib.
- 6 Run the EDA checklist before writing a single line of training code.

Next module

With data in hand, we implement two classic algorithms from scratch: KNN and K-Means, entirely in PyTorch.

Questions?

See you in Module 4.